Project 4: Extending the pipeline with compute cores

sbhatnagar45@gatech.edu

Task 1: Naive Compute Core Implementation

1.1 Implementation Details

The first task involved extending the Macsim simulator to handle non-memory compute instructions. The core logic was to identify these instructions and model their execution latency.

First, the simulator was enabled to distinguish compute instructions from other operations by checking each instruction's opcode against a predefined list. Within the core's main execution loop, logic was added to handle these newly identified compute instructions. When a compute instruction is fetched, it is not executed immediately but is instead placed into an in-flight execution buffer. This buffer tracks instructions that are "executing," with each being assigned a completion timestamp. The core stalls if it attempts to issue a new compute instruction when this buffer is already full, simulating a limited number of execution units. At the start of each cycle, the simulator clears any instructions from the buffer whose completion time has passed, making their results available. For this initial task, all compute instructions had a default latency of 1 cycle.

2. Task 2: Tensor Core Implementation

1.2 Implementation Details

Task 2 extended the compute core to simulate a more powerful Tensor Core, characterized by a wider execution path, specialized instruction latencies, and hazard-aware scheduling. To model the specialized hardware, the concept of dynamic instruction latency was introduced. The simulator now inspects the opcode of each compute instruction. If it matches the pattern for a tensor operation (e.g., starting with "H*"), a high latency value (64 cycles) read from the configuration is assigned. This was paired with support for a wider execution path, allowing multiple high-latency instructions to be in-flight concurrently, with the width also being configurable. The most critical part of this task was implementing a mechanism to prevent data hazards (the dependency check), which become significant with long-latency instructions. A dependency check was added to the warp scheduler. Before a warp can be scheduled, the scheduler inspects the source registers of its next instruction. It then checks if any instruction currently in the execution buffer from the same warp is writing to one of those source registers. The Round-Robin scheduler was revised to be hazard-aware. It now iterates through the pool of available warps and schedules the first one that passes the dependency check. If all available warps are stalled due to dependencies, the core does not schedule any work for that cycle, correctly modeling the pipeline stall.

3. Performance Analysis

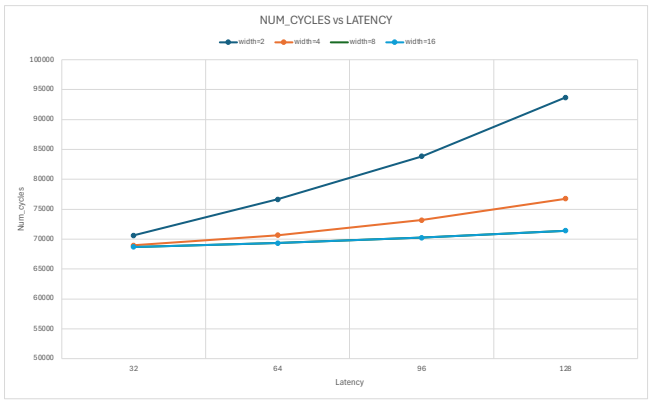
3.1. Performance Comparison: Compute Core vs. Tensor Core

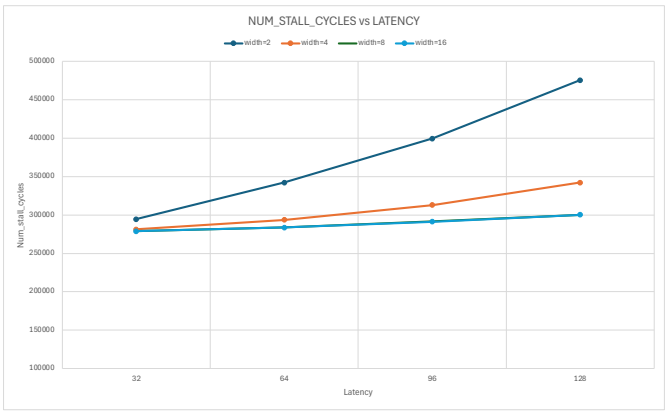
This section compares the performance of the baseline Compute Core (Task 1) running the *gemm_float* trace against the Tensor Core (Task 2) running the *gemm_half* trace. The results clearly show that the specialized Tensor Core architecture provides a significant performance improvement for this workload. The key metrics for each task also match the reference numbers provided for the project, confirming the correctness of the implementations.

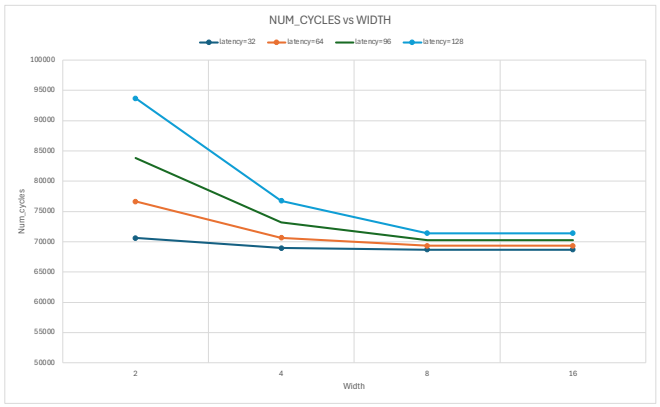
Metric	Task 1: CC reference (gemm_float)	Task 1: CC actual (gemm_float)	Task 2: TC reference (gemm_half)	Task 2: TC actual (gemm_half)
NUM_CYCLES	109220.0	109220.0	69344.0	69344.0
NUM_INSTRS_RETIRED	485376.0	485376.0	254208.0	254208.0
NUM_STALL_CYCLES	362254.0	362254.0	283688.0	283688.0
INSTR_PER_CYCLE	4.444021	4.444021	3.665898	3.665898

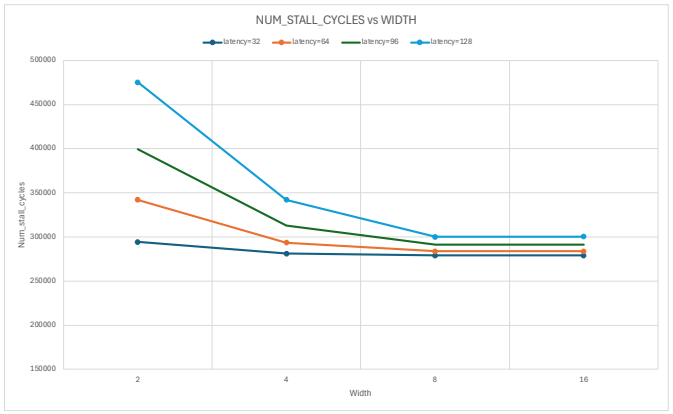
The Tensor Core implementation is approximately 1.58x faster than the Naive Compute Core, reducing the total execution time from over 109,000 cycles to just over 69,000. The primary reason for this speedup is the power of hardware specialization. The *gemm_half* trace leverages powerful HMMA (Half-precision Matrix Multiply-Accumulate) instructions. As a result, the Tensor Core needed to execute nearly 50% fewer instructions to complete the same amount of work. Interestingly, the Tensor Core's Instructions-Per-Cycle (IPC) is lower than that of the Compute Core (3.67 vs. 4.44). This is an expected trade-off. The high latency of tensor instructions and the necessary stalls for hazard prevention (reflected in the Stall Cycles metric) mean that instructions complete at a slower rate. However, because each instruction accomplishes significantly more work, the overall task throughput is much higher. This demonstrates that for specialized workloads, total execution time is a more important metric than the raw instruction completion rate.

3.2. Parameter Sweeping Analysis (Task 2) To understand the architectural trade-offs of the Tensor Core, its performance (NUM_CYCLES, NUM_STALL_CYCLES) was evaluated across a range of tensor instruction latencies (32-128 cycles) and execution widths (2-16). Figure 1 below:









Analysis of Latency: As shown in Figure 1(top-row), for any given execution width, increasing the latency of tensor instructions leads to a near-linear increase in total execution cycles. This is expected, as the gemm_half workload is dominated by these long-running operations. Figure 1 (top-right) confirms this, showing that stall cycles also increase with latency. This is because the scheduler must wait longer for instructions to complete, increasing the probability of a subsequent instruction in the same warp having a register dependency.

Analysis of Execution Width: Figure 1 (bottom-row) clearly demonstrates the benefit of parallelism. Increasing the execution width allows the core to process more instructions simultaneously, which consistently reduces the total execution time (num_cycles). However, the data also reveals a clear point of diminishing returns. The performance gain from increasing the width from 2 to 4, and from 4 to 8, is significant. In contrast, the improvement from 8 to 16 is minimal. For latencies of 32 and 64, there is no performance improvement at all when moving from a width of 8 to 16. This indicates that with an execution width of 8, the core can already exploit most of the available instruction-level parallelism in this specific workload. Beyond this point, the scheduler struggles to find enough independent instructions to fill the wider execution path, and the extra hardware provides little benefit.

4. Discussion on Warp Scheduling Strategies

The Round-Robin (RR) scheduler implemented in this project is simple and fair, but not the most efficient. For a Tensor Core architecture with a mix of long-latency (tensor) and short-latency (memory, simple compute) instructions, more advanced scheduling strategies could yield better performance.

- **Greedy-Then-Oldest (GTO)**: GTO scheduler prioritizes warps that are ready to issue memory requests, aiming to hide memory latency by keeping the memory subsystem busy. In a compute-bound workload like GEMM, this might be less effective. However, if a warp is stalled on a long compute operation, GTO could be beneficial by scheduling other warps to perform memory operations in the meantime, overlapping compute and memory access.
- **Cache-Conscious Wavefront Scheduling (CCWS)**: This strategy prioritizes warps that are likely to hit in the cache, aiming to maximize locality and minimize memory stalls. This would be highly effective in workloads with predictable memory access patterns. For the Tensor Core, a CCWS-like approach could be adapted to prioritize warps that are not dependent on long-latency compute instructions, effectively scheduling the easy work first while the long operations complete in the background. This could improve core utilization by reducing stalls caused by register dependencies.

In summary, while dependency-aware RR scheduler prevents hazards, a more advanced scheduler could further improve performance by intelligently prioritizing warps based on their instruction type or predicted cache behavior to better hide the long latency of tensor operations.